



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

APAI LAB 02: DNN Definition and Training

Călin Diaconu – calin.diaconu2@unibo.it

Alberto Dequino – alberto.dequino@unibo.it

Ahmad Mirsalar – seyedahmad.mirsalar2@unibo.it

Ihor Sachok – ihor.sachok@unibo.it



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

01. Lab Information

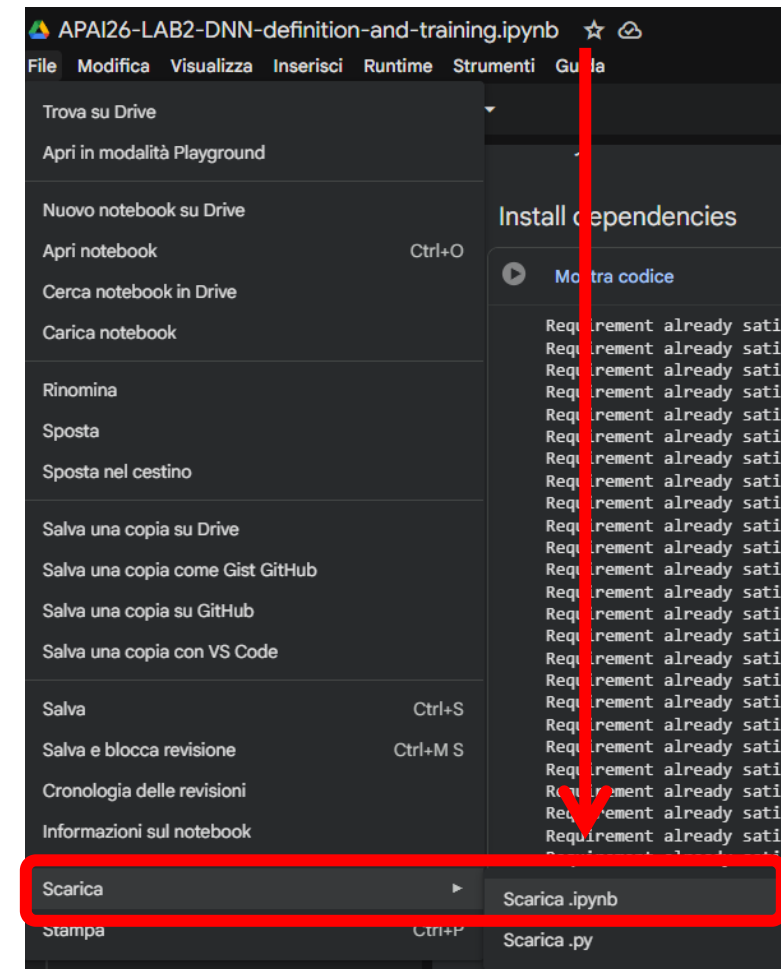
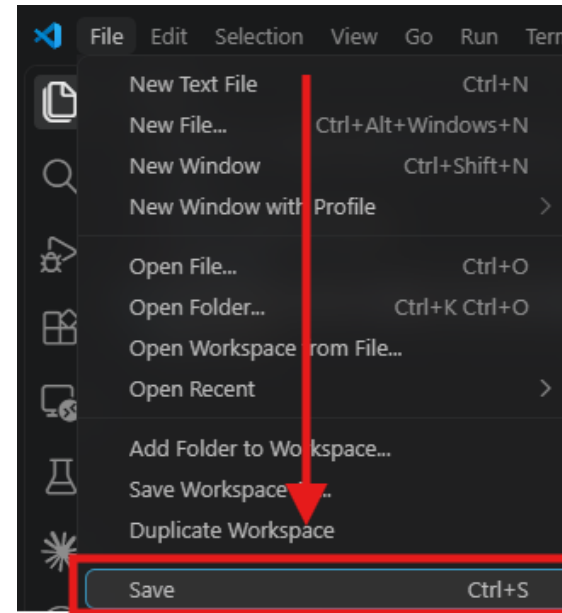
01. Lab Information – Objective of the Class

- **Intro:** building blocks of CNNs (convolution, batch norm, ReLU, softmax, max pooling, dropout)
- **Tasks:** define and train a simple CNN with PyTorch:
 - Build the network from the layers introduced in the intro
 - Train the network on a dataset
 - Complete Tasks 01-06 in the Python Notebook
- **Programming language:** Python
- **Lab duration:** 3h
- **AI policy:** we will not prevent you from using AI in solving the assignments, but the lab is meant to be a hands-on learning experience. To keep it useful, we have included an .md file (also used in a [Stanford class](#)), that should be automatically read by AI agents, in order to prevent them to give you a direct solution and rather guide you towards finding it yourself.
- The class is meant to be interactive: coding together and on your own!



01. Lab Information – How to Deliver the Assignment

- You will solve the assignment inside the given Python notebook and deliver the notebook on Virtuale
 - **Run the cells and save the notebook**, so the results show up in the cells
 - **Rename the file** to `LAB<number_of_the_lesson>_APAI_<name_surname>.ipynb`
 - **If you run locally**, upload the saved notebook directly from your PC
 - **If you run on Colab**: download it first (*File* → *Download* → *Download .ipynb*), then upload it
 - Use **Virtuale** to upload the notebook before the deadline (Sunday 25th of October, end of day)
 - For each lab that you solve and submit successfully, you will receive **1 extra point**, up to a **maximum of 4 points**



LAB 01 - PULP Embedded Programming

Opened: Friday, 2 October 2026, 12:00 AM
Due: Sunday, 25 October 2026, 11:59 PM

Upload your completed PDF assignment here for LAB01.

Add submission

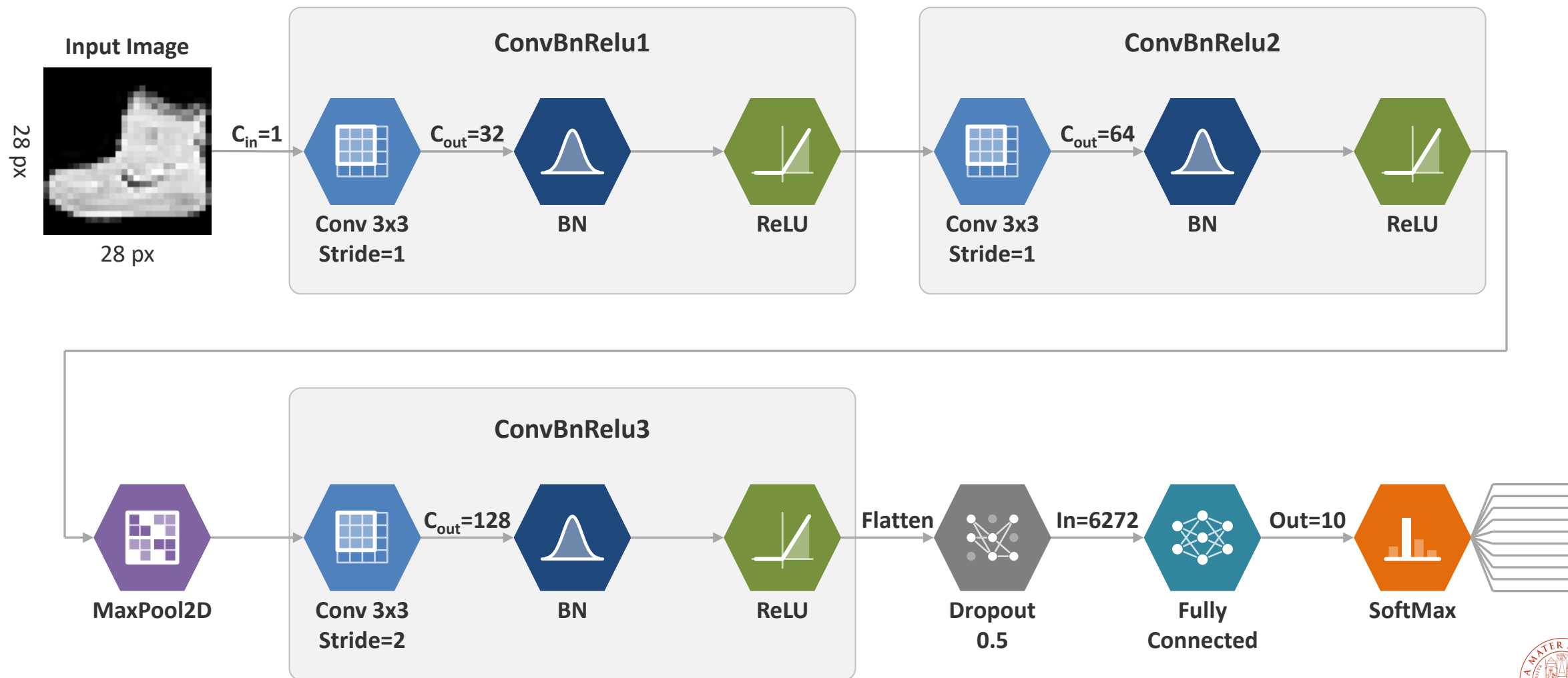




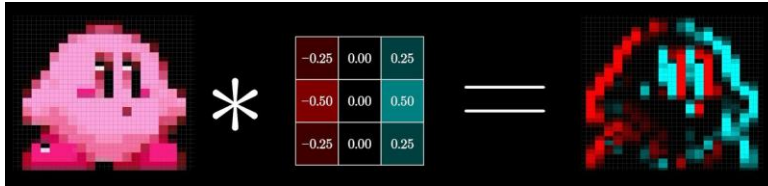
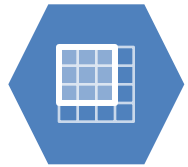
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

02. Introductory Information

02. Intro Information – Convolutional Neural Networks (CNNs)



02. Intro Information – 2D Convolutions



Slides **learnable kernels** over the input to detect **local features** (edges, textures)

- **Kernel:** size of the convolution kernel (height, width)
- **Channels:** channels of the input / output feature map
- **Stride:** how many input pixels the kernel skips as it slides
- **Padding:** how many zeros are added on the border of the input feature map

Output feature map size

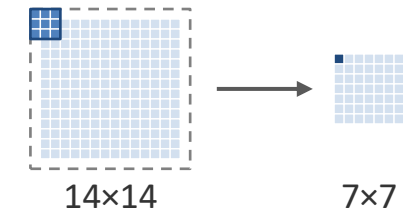
$$H_{\text{out}} = \lfloor (H_{\text{in}} + 2P_H - K_H) / S_H \rfloor + 1$$
$$W_{\text{out}} = \lfloor (W_{\text{in}} + 2P_W - K_W) / S_W \rfloor + 1$$

K = kernel size, P = padding, S = stride

Example: Conv3 in prev. slide

$$H_{\text{in}} = 14, K = 3, P = 1, S = 2$$

$$H_{\text{out}} = \lfloor 13 / 2 \rfloor + 1 = 7$$



Visualizer: ezyang.github.io/convolution-visualizer

02. Intro Information – BN, ReLU, and Softmax



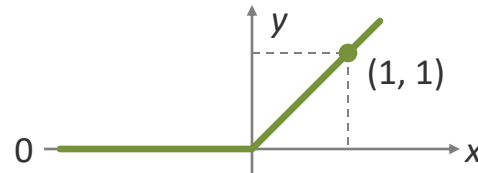
Batch Normalization

- Purpose: **Stabilize and accelerate** the **training** of DNNs by **re-centering** and **re-scaling** the intermediate layer **activations** across **mini-batches**
- Input and output sizes match: NCHW **shape unchanged**
- **num_features**: channels of the input feature map



ReLU

- **Non-linear activation** function - Rectified Linear Unit
- Purpose: prevent the **vanishing gradient problem**
- $\text{ReLU}(x) = \max(0, x)$



SoftMax

- **Activation** that transforms the input **feature** array into a **probability** array

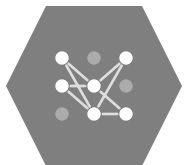
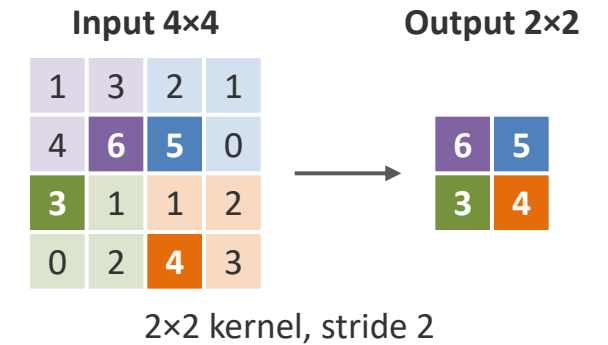
Input		Probabilities
1.0	→	0.23
-1.0		0.03
2.0		0.64
0.1		0.10
		Sum = 1

02. Intro Information – Max Pooling & Dropout



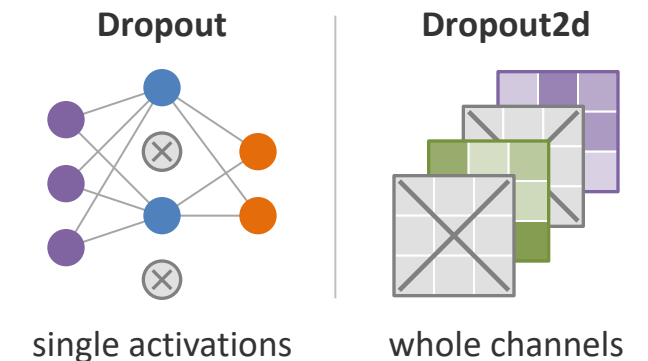
Max Pooling

- Only selects the **maximum value** in a given window, with the purpose of keeping the **most relevant features**
- **Reduces the size of the input** feature map according to **stride and kernel size**



Dropout

- Layer used to **prevent overfitting**
- **Zeroes random activations** with probability p , since some units may learn **features that don't generalize well**
- **Dropout2d** zeroes **whole channels** of the feature map





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

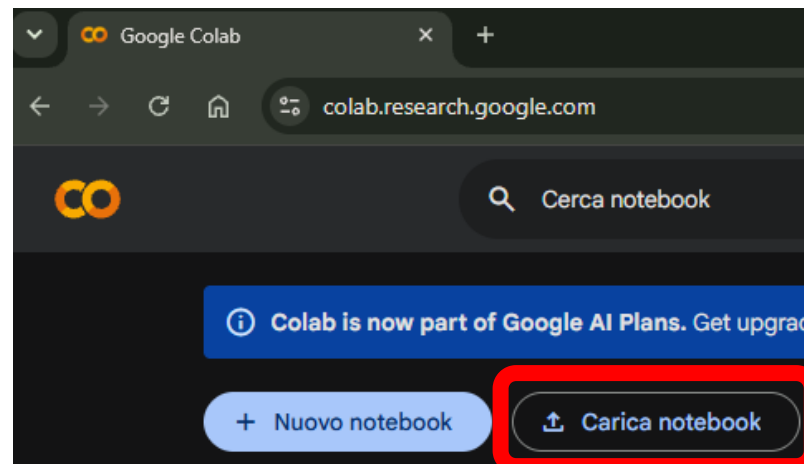
03. Setup

03. Setup – Google Colab

- **THIS LAB MUST BE RUN ON YOUR OWN PERSONAL COMPUTER**, since the lab machines have very limited internet access, which wouldn't allow you to access either Google Colab, or to download the necessary libraries
- There are 2 paths of completing the assignment
 - Running locally, inside VS Code – open the notebook in VS Code and it should work with minimal setup; VS Code will direct you to install the missing components if that's the case
 - Running on Google Colab – an online tool for running Python Notebooks, on cloud, accessing it via your browser (so it should be universally compatible), following the steps below

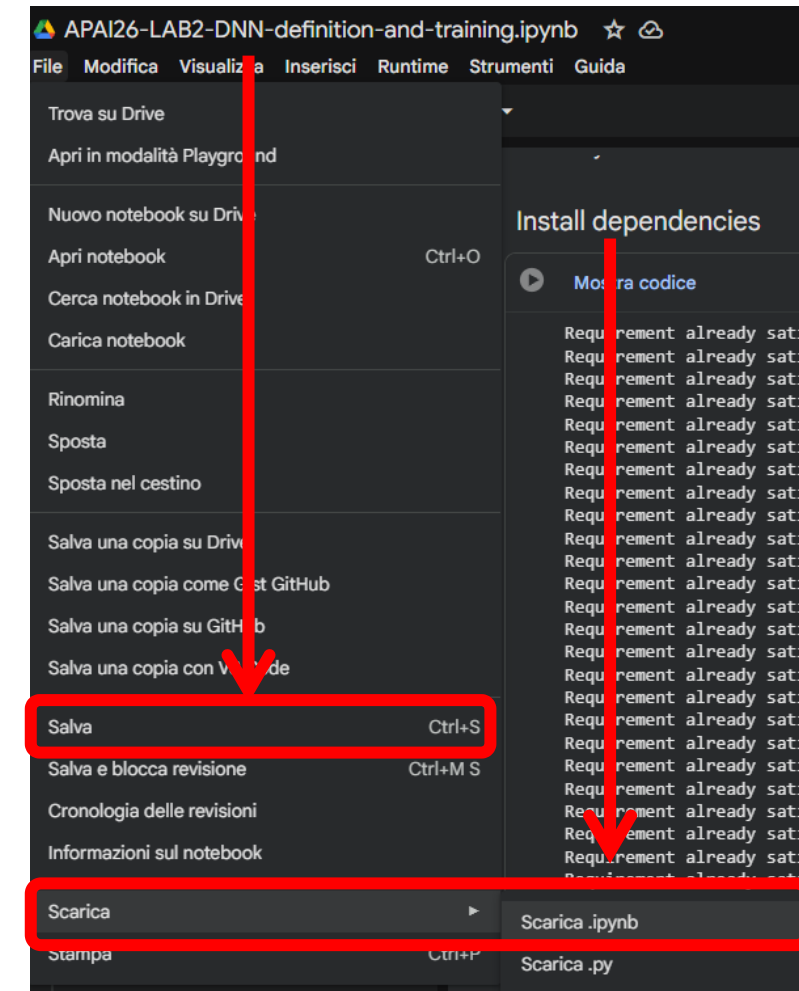
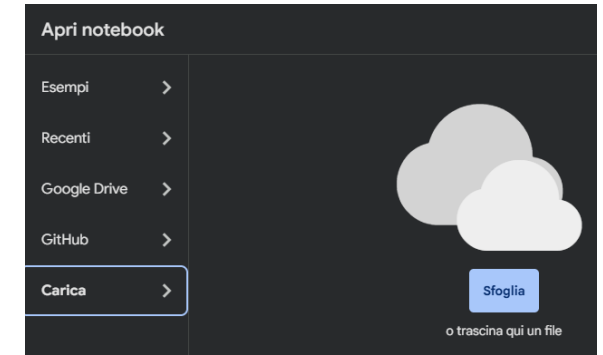
Colab steps:

- Step 1: Download the Notebook, either from Virtuale, or clone from [the repo](#)
- Step 2: Access [Google Colab](#) and choose the “Upload Notebook”/”Carica notebook” option



03. Setup – Google Colab

- Step 3: Use the pop-up to upload the notebook that you just downloaded
- Step 4: Go through the tasks and requirements, and solve the assignments
 - You will have to follow some usual steps for training a DNN with PyTorch:
 - Define the architecture
 - Evaluate its complexity
 - Prepare the dataset and the data loader
 - Prepare the evaluation functions
 - Run the actual training loop
 - Evaluate the trained model and make sure that you can save it and load it
 - If you have questions at any point, feel free to ask!
- Step 5: Once you are happy with the results and have completed all tasks, be sure you do NOT delete the content of any of the run cells and that everything is properly saved.
- Step 6: Now, follow the Colab download instructions from the first part and upload the completed Notebook to Virtuale





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Călin Diaconu – calin.diaconu2@unibo.it

Alberto Dequino – alberto.dequino@unibo.it

Ahmad Mirsalar – seyedahmad.mirsalar2@unibo.it

Ihor Sachok – ihor.sachok@unibo.it